

Resumen Completo del Curso

Google IT Automation with Python Certificate

8 vídeos • Pensado para principiantes en programación

Bootcamp AI Engineering

De qué trata este curso

Este curso de Google en YouTube enseña a usar **Python para automatizar tareas de informática**: gestionar archivos, procesar texto, trabajar con el sistema operativo Linux, y escribir scripts que hagan trabajo repetitivo por ti. Está diseñado para personas que quieren trabajar en IT (soporte técnico, administración de sistemas).

 *Consejo del profesor: si en algún momento te sientes perdida, no te agobies. Puedes ver los vídeos todas las veces que necesites. Lo más importante es practicar los ejemplos en tu propio ordenador.*

Vídeo 1 — Instalar Python en Windows, Mac y Linux

¿Qué es el sistema operativo?

Antes de instalar Python, el profesor explica qué es el sistema operativo (SO). Es el programa que gestiona TODO lo que pasa en el ordenador: lee y escribe archivos, decide cómo los programas se inician y se terminan, reparte la memoria entre los programas, y controla el hardware. El profesor lo describe como **"el jefe total que no se anda con tonterías"**.

El SO tiene dos partes:

- **Kernel:** el núcleo. Habla directamente con el hardware (procesador, memoria, disco duro). Nosotros como usuarios nunca lo tocamos directamente.
- **Espacio de usuario:** todo lo que usamos nosotros directamente: los programas, la terminal, la interfaz gráfica.

 *Cuando el profesor dice 'sistema operativo', se refiere a ambas partes juntas: el kernel y el espacio de usuario.*

Los sistemas operativos más importantes

Sistema operativo	Quién lo hace	Para qué se usa
Windows	Microsoft	Ordenadores personales y empresas. La mayoría de PCs lo traen instalado.
macOS	Apple	Ordenadores Apple (MacBook, iMac). Viene preinstalado.

Sistema operativo	Quién lo hace	Para qué se usa
Linux	Comunidad open source	Servidores profesionales. La MAYORÍA de los servidores del mundo usan Linux.
Android	Google	Móviles. ¡También tiene un kernel Linux por dentro!

💡 *Linux es open source (código abierto): es gratis, se puede modificar y distribuir libremente. Por eso existen muchas versiones distintas llamadas distribuciones, como Ubuntu, Debian o Red Hat.*

¿Por qué Python funciona en todos los sistemas operativos?

Python es un lenguaje **multiplataforma**. Puedes escribir el mismo script en Windows y ejecutarlo sin cambios en Linux o Mac. Eso lo hace perfecto para IT, donde a menudo trabajas con distintos sistemas.

¿Cómo saber si ya tienes Python instalado?

Abre la terminal (o símbolo del sistema en Windows) y escribe:

```
python --version
```

Pueden pasar tres cosas:

- El ordenador dice que no reconoce el comando → Python no está instalado.
- Te muestra Python 2.x → Tienes instalada una versión antigua. Necesitas Python 3.
- Te muestra Python 3.x → ¡Todo bien! Ya tienes lo que necesitas.

⚠ *Este curso usa Python 3. Python 2 es una versión antigua y algunas cosas son diferentes. Si tienes Python 2, intenta también: `python3 --version`*

Cómo instalar Python

En Windows

- Descarga el instalador desde python.org
- Ejecútalo como administrador
- **MUY IMPORTANTE:** Antes de hacer clic en 'Install Now', marca la casilla **"Add Python to PATH"**. Sin esto, Python no se encontrará desde la terminal.

En macOS

- macOS viene con Python 2 instalado, pero necesitas Python 3
- Descarga el instalador desde python.org o usa Homebrew
- En Mac el comando es `pip3` (no `pip`) para instalar módulos

En Linux

- Casi seguro que ya tienes Python. Compruébalo con `python3 --version`
- Si no lo tienes, en Ubuntu/Debian: `sudo apt install python3`

¿Qué es pip y para qué sirve?

pip es la herramienta para instalar módulos externos (librerías adicionales de Python). Cuando Python viene instalado, trae los módulos más básicos, pero hay miles de módulos adicionales para hacer cosas más específicas: generar PDFs, enviar emails, analizar datos, trabajar con imágenes, etc.

Todos estos módulos se publican en **PyPI** (el índice de paquetes de Python, pypi.org). Para instalar uno:

```
pip install nombre_del_modulo      # En Windows
pip3 install nombre_del_modulo     # En Mac y Linux
```

El profesor muestra tres ejemplos reales de módulos:

- **requests:** para descargar páginas web o hablar con APIs.
- **arrow:** para manejar fechas y horas de forma sencilla.
- **pandas:** para analizar datos (muy usado en ciencia de datos).

💡 *Ejemplo del profesor: si necesitas escribir una automatización que cambia el tamaño de imágenes, instalarías el módulo Pillow (PIL). Si necesitas analizar los errores de un servidor web y generar estadísticas, usarías pandas.*

Vídeo 2 — Cómo ejecutar un script Python

Lenguajes compilados vs. interpretados — ¿cuál es la diferencia?

Esta es una explicación fundamental del profesor. Hay dos grandes tipos de lenguajes de programación:

Tipo	¿Cómo funciona?	Ejemplos	¿Ventaja?
Compilado	El código se traduce ENTERO a lenguaje de máquina ANTES de ejecutarse. Necesitas compilar para cada sistema operativo.	C, C++, Go, Rust	Muy rápido al ejecutar
Interpretado	El código se ejecuta LÍNEA A LÍNEA en el momento, sin compilar antes.	Python, JavaScript, Bash	Más fácil de desarrollar y funciona en cualquier SO
Mixto	Se compila a código intermedio que funciona en varios sistemas.	Java, C#	Lo mejor de ambos mundos

💡 *Analogía: un lenguaje compilado es como traducir un libro entero antes de leerlo. Un lenguaje interpretado es como tener un traductor simultáneo que va leyendo frase a frase en tiempo real.*

Python es interpretado. Eso significa que puedes escribir código en Windows y ejecutarlo en Linux **sin cambiar nada**. El intérprete de Python (que sí es específico de cada SO) se encarga de la traducción.

Tres formas de ejecutar Python

Forma 1: Modo interactivo (para experimentar)

Escribe `python3` en la terminal y se abre una especie de calculadora donde puedes escribir código y ver el resultado inmediatamente:

```
python3
>>> print('Hola')
Hola
>>> 2 + 2
4
```

⚠ *El problema del modo interactivo: cuando cierras la terminal (con `exit()` o `Ctrl+D` en Linux/Mac, `Ctrl+Z` en Windows), TODO el código que escribiste desaparece. No se guarda nada.*

Forma 2: Ejecutar un archivo .py (la forma normal)

Los scripts se guardan en archivos con extensión `.py` y se ejecutan así:

```
python3 mi_script.py
```

Forma 3: Ejecutar el archivo directamente (solo Linux y Mac)

Puedes hacer que el archivo se ejecute solo, sin escribir `python3` cada vez. Para eso necesitas dos cosas:

Paso 1: Añade el **shebang** en la primera línea del archivo. Esta línea le dice al sistema operativo con qué programa ejecutar el script:

```
#!/usr/bin/env python3
print('Hola mundo')
```

Paso 2: Dale permiso de ejecución al archivo con el comando `chmod`:

```
chmod +x mi_script.py
```

Ahora puedes ejecutarlo así:

```
./mi_script.py
```

💡 *¿Por qué `./`? El punto (`.`) significa 'la carpeta actual'. Si no lo pones, el sistema busca el programa en otras carpetas del sistema y no encuentra el tuyo.*

Reutilizar código: crear tus propios módulos

Cuando un script crece, es buena idea separar el código en archivos distintos para reutilizarlo. Imagina que tienes dos scripts distintos que ambos necesitan enviar emails. En lugar de copiar y pegar el código de emails en los dos archivos (lo cual crea problemas cuando necesitas arreglarlo: ¡tendrías que arreglarlo en dos sitios!), mejor lo pones en un módulo aparte.

El profesor muestra el ejemplo del módulo **areas.py**:

Ejemplo: Crear y usar un módulo propio

```
# archivo: areas.py
import math

def circulo(radio):
    return math.pi * radio ** 2
```

```
def triangulo(base, altura):  
    return base * altura / 2  
  
# En otro archivo, lo importas así:  
import areas  
  
print(areas.circulo(5))      # 78.53...  
print(areas.triangulo(3, 4)) # 6.0
```

Editores de código — ¿cuál usar?

Para escribir código necesitas un editor. El profesor menciona varias opciones:

- **VS Code:** el más recomendado para el bootcamp. Tiene autocompletado, colores de sintaxis y muchas extensiones.
- **PyCharm:** muy completo, ideal para proyectos grandes.
- **Nano o Vim:** editores de terminal. Importantes para cuando trabajas en servidores remotos sin interfaz gráfica.

Dos características importantes de los buenos editores:

- **Syntax highlighting (colores de sintaxis):** el editor reconoce el lenguaje y colorea las diferentes partes del código. Por ejemplo, las palabras reservadas como `def` o `return` aparecen en un color distinto.
- **Code completion (autocompletado):** cuando empiezas a escribir, el editor sugiere cómo completar la palabra. Por ejemplo, escribes `ret` y el editor sugiere `return`.

Automatización: ¿cuándo merece la pena automatizar algo?

El profesor explica que automatizar no siempre es la mejor opción. Aquí está la regla para decidir:

 **Regla del profesor:** Si el tiempo que tardarás en escribir el script es **MENOR** que el tiempo total que ahorrarás no haciendo la tarea manualmente, entonces automatiza.

Ejemplo concreto del vídeo:

- Generar un informe diario te lleva 5 minutos cada día.
- Crear la automatización te lleva 1 hora (60 minutos).
- $60 \text{ minutos} \div 5 \text{ minutos/día} = 12 \text{ días}$ → A partir del día 12, ya estás ahorrando tiempo.
- Si en 6 meses cambian los requisitos del informe, puede que ya no haya merecido la pena.

El profesor también menciona el **principio de Pareto** aplicado a IT: el 20% de las tareas que haces representan el 80% de tu trabajo. Si identificas y automatizas ese 20%, ahorras muchísimo tiempo.

Peligros de la automatización mal hecha

El profesor dedica tiempo a explicar qué puede salir mal con la automatización:

- **Fallos silenciosos:** la automatización falla pero nadie se entera. El ejemplo del vídeo: un sistema de copias de seguridad que deja de funcionar sin avisar. Cuando el servidor se cae y necesitas restaurar los datos, descubres que no hay copias. ¡Desastre!

- **'Bit rot' (código que se pudre):** el sistema cambia (por ejemplo, un disco duro cambia de nombre) pero la automatización no se actualiza. El script empieza a fallar porque asume cosas que ya no son ciertas.
- **Automatización que hace lo incorrecto:** peor que no funcionar es funcionar mal. Por ejemplo, un sistema de backups que hace copias del conjunto de datos equivocado.

Soluciones que propone el profesor:

- Añadir notificaciones: si la automatización falla, que envíe un email o una alerta.
- Hacer tests periódicos: comprobar regularmente que la automatización está haciendo lo correcto.
- Escribir logs: que el script registre lo que hace para poder investigar problemas.

Ejemplo práctico: script de salud del ordenador

El profesor escribe un script que comprueba si el ordenador está bien usando dos módulos de la librería estándar:

Ejemplo: Health check — comprobar disco y CPU

```
#!/usr/bin/env python3
import shutil
import psutil

def check_disk_usage(disk):
    du = shutil.disk_usage(disk)
    free = du.free / du.total * 100
    return free > 20    # True si hay más de 20% libre

def check_cpu_usage():
    usage = psutil.cpu_percent(1)    # promedio en 1 segundo
    return usage < 75    # True si CPU usa menos del 75%

if not check_disk_usage('/') or not check_cpu_usage():
    print('ERROR: El sistema no está bien.')
else:
    print('Todo OK.')
```



Vídeo 3 — Manejo de archivos en Python

¿Por qué necesitamos trabajar con archivos?

Hasta ahora, cuando un script necesitaba datos, los escribíamos directamente en el código (variables). Eso funciona para cosas pequeñas, pero cuando tienes que procesar muchos datos —como los registros de un servidor o una lista de empleados— es mucho mejor leerlos desde un archivo.

Leer un archivo: la forma básica

El profesor usa de ejemplo un archivo llamado **spider.txt** con la canción Itsy Bitsy Spider. Para abrirlo y leerlo:

Ejemplo: Abrir y leer un archivo línea a línea

```
file = open('spider.txt')    # Abrimos el archivo
```

```
# readline() lee UNA línea cada vez
# Cada vez que lo llamas, avanza a la siguiente
print(file.readline()) # Imprime la línea 1
print(file.readline()) # Imprime la línea 2

# read() lee todo lo que queda desde donde estás
print(file.read())      # Imprime líneas 3, 4, 5, 6

file.close()           # ¡Hay que cerrar el archivo!
```

¿Por qué hay que cerrar los archivos?

El profesor explica que cerrar los archivos es importante por tres razones:

- **Bloqueo del archivo:** mientras el archivo está abierto en tu script, otros programas no pueden usarlo.
- **Límite de archivos abiertos:** el sistema tiene un límite de cuántos archivos puede tener abiertos a la vez. Si abres muchos en un bucle sin cerrarlos, el sistema se queda sin recursos.
- **Race conditions:** si varios procesos intentan modificar el mismo archivo al mismo tiempo sin cerrar correctamente, pueden pasar cosas raras e impredecibles.

La forma recomendada: el bloque 'with'

El profesor confiesa: *"yo soy el peor cerrando archivos. Se me olvida constantemente."* Por eso Python ofrece una forma que cierra el archivo automáticamente: el bloque **with**:

Ejemplo: Con el bloque 'with' (recomendado)

```
with open('spider.txt') as file:
    print(file.readline()) # Lee línea 1
    print(file.readline()) # Lee línea 2

# Al salir del bloque 'with', el archivo se cierra solo.
# No necesitas llamar a file.close()
```

 **Piénsalo como:** el bloque 'with' es como pedir un libro en la biblioteca. Cuando terminas de leer (al salir del bloque), el libro se devuelve automáticamente sin que tengas que recordarlo.

Iterar el archivo línea a línea y el problema de las líneas vacías

Puedes recorrer un archivo como si fuera una lista:

Ejemplo: Problema de líneas vacías al iterar

```
# Esto puede imprimir líneas vacías entre el contenido:
with open('spider.txt') as file:
    for line in file:
        print(line.upper()) # Muestra texto EN MAYÚSCULAS

# ¿Por qué aparecen líneas vacías?
# Cada línea del archivo ya tiene \n al final.
# print() añade otro \n. Resultado: doble salto de línea.

# Solución: usar strip() para eliminar espacios y saltos de línea:
with open('spider.txt') as file:
    for line in file:
        print(line.strip().upper()) # Ahora funciona bien
```

Modos de apertura de archivos

Cuando abres un archivo puedes decirle a Python qué quieres hacer con él:

Modo	Qué significa	¿Qué pasa si el archivo ya existe?
"r"	Solo lectura (por defecto, no hace falta ponerlo)	Lo abre tal cual
"w"	Solo escritura	¡BORRA TODO el contenido existente!
"a"	Añadir al final (append)	Conserva el contenido y añade al final
"r+"	Lectura y escritura	Lo abre tal cual, puedes hacer las dos cosas

✗ PELIGRO: Si abres un archivo en modo 'w' y el archivo ya tenía contenido, lo pierdes para siempre. El profesor lo repite dos veces: 'esto ha arruinado a mucha gente más de una vez'. Comprueba siempre qué archivo estás abriendo.

Ejemplo de escritura:

Ejemplo: Escribir en un archivo nuevo

```
with open('mi_novela.txt', 'w') as file:
    file.write('Era una noche oscura y tormentosa.') # Escribe el texto
    # El método write() devuelve el número de caracteres escritos

# Para añadir contenido al final sin borrar lo anterior:
with open('mi_novela.txt', 'a') as file:
    file.write('\nEl detective entró en la habitación.')
```

Gestionar archivos con el módulo os

El módulo **os** te permite hacer desde Python las mismas cosas que harías en la terminal. La ventaja es que el módulo **os** funciona igual en Windows, Mac y Linux (aunque los separadores de ruta sean distintos: / en Linux/Mac, \ en Windows).

Ejemplo: Operaciones con archivos usando os

```
import os

# Ver en qué carpeta estás
print(os.getcwd()) # /home/usuario/proyectos

# Borrar un archivo
os.remove('borrar_esto.txt')

# Renombrar un archivo
os.rename('viejo.txt', 'nuevo.txt')

# Crear una carpeta
os.mkdir('nueva_carpeta')

# Listar los archivos de una carpeta
print(os.listdir('.')) # [archivo1.py, archivo2.txt, ...]

# Antes de borrar, comprobar si el archivo existe:
if os.path.exists('mi_archivo.txt'):
    os.remove('mi_archivo.txt')
else:
    print('El archivo no existe')
```


Información sobre archivos

Con el submódulo **os.path** puedes obtener información sobre los archivos:

Ejemplo: Obtener información de un archivo

```
import os
from datetime import datetime

# Tamaño del archivo en bytes
tamano = os.path.getsize('mi_archivo.txt')
print(f'Tamaño: {tamano} bytes')

# Fecha de última modificación
timestamp = os.path.getmtime('mi_archivo.txt')
# El timestamp es un número raro (segundos desde 1 enero 1970)
# Lo convertimos a fecha legible:
fecha = datetime.fromtimestamp(timestamp)
print(f'Modificado: {fecha}')

# Ruta completa del archivo (en lugar de solo el nombre)
ruta_completa = os.path.abspath('mi_archivo.txt')
print(ruta_completa)      # /home/usuario/proyectos/mi_archivo.txt
```

 *El profesor explica el Unix timestamp: es el número de segundos desde el 1 de enero de 1970. Ese día se eligió porque fue cuando se empezó a publicar Unix. Como informático verás este número constantemente en logs, bases de datos, etc.*

Archivos CSV: datos en tablas

CSV significa Comma Separated Values (Valores Separados por Comas). Es como una hoja de cálculo guardada en texto plano. Cada línea es una fila y las comas separan las columnas:

Ejemplo: Ejemplo de archivo CSV de empleados

```
# empleados.csv
Sabrina Green,802-867-5309,Encargada de sistemas
Eli Jones,684-3481279,Administrador de bases de datos
Melody Daniels,846-687-7436,Soporte técnico
```

Python tiene un módulo llamado **csv** para leer y escribir estos archivos fácilmente:

Ejemplo: Leer un CSV y acceder a sus datos

```
import csv

with open('empleados.csv') as file:
    lector = csv.reader(file)
    for fila in lector:
        # fila es una lista: ['Sabrina Green', '802-867-5309', 'Encargada...']
        nombre, telefono, rol = fila    # Desempaquetamos los valores
        print(f'Nombre: {nombre}, Rol: {rol}')
```

Si el CSV tiene una primera fila con los nombres de las columnas, puedes usar **DictReader** para acceder a los datos por nombre de columna en lugar de por posición:

Ejemplo: Leer CSV con DictReader (más legible)

```
import csv

# El CSV tiene cabecera: nombre,usuarios,estado
with open('programas.csv') as file:
    lector = csv.DictReader(file)
    for fila in lector:
        # Accedemos por nombre de columna, no por índice [0], [1]...
```

```
print(f'{fila["nombre"]} tiene {fila["usuarios"]} usuarios')

# El profesor menciona un programa llamado 'Chatty Chicken'
# que solo tenía 4 usuarios. ¡Con un nombre así, será un éxito!
```

Vídeo 4 — Expresiones regulares en Python

¿Qué son las expresiones regulares?

Una expresión regular (también llamada **regex** o regexp) es un patrón para buscar texto. Te permite hacer búsquedas mucho más avanzadas que simplemente buscar una palabra exacta.

 *El profesor dice: 'Sé que esto es complejo. No voy a mentiros. Si en algún momento os sentís perdidos, podéis volver a ver los vídeos. Y practicad los ejemplos.'*

¿Para qué sirven? Puedes responder preguntas como:

- ¿Cuáles son todas las palabras de cuatro letras en este archivo?
- ¿Cuántos tipos distintos de error hay en este log de servidor?
- ¿Qué líneas contienen una dirección de email?
- ¿Qué líneas empiezan con una fecha en formato DD/MM/YYYY?

 *El profesor (que es administrador de sistemas) dice: 'Yo uso regex cuando necesito extraer información de un archivo que tiene mucho contenido que no me interesa. Por ejemplo, si tengo un archivo de configuración de servidores y solo quiero los nombres de los servidores, escribo una regex que elimina todo lo demás y me queda solo lo que necesito.'*

Primer ejemplo: buscar texto con grep

Antes de entrar en Python, el profesor muestra las regex con el comando **grep** de Linux, que es más visual para empezar. grep muestra las líneas que coinciden con el patrón:

Ejemplo: Usando grep para buscar patrones

```
# Buscar todas las palabras que contengan 'thon'
grep 'thon' /usr/share/dict/words
# Resultado: python, marathon, python... (con la parte 'thon' resaltada)

# El PUNTO (.) en regex significa 'cualquier carácter'
# El patrón 'l.rts' encuentra:
grep 'l.rts' /usr/share/dict/words
# Resultado: alerts, blurts, flirts
# El punto se sustituyó por 'e', 'u', 'i' respectivamente

# El CIRCUNFLEJO (^) = inicio de línea
grep '^fruit' /usr/share/dict/words
# Solo palabras que EMPIEZAN por 'fruit'

# El DÓLAR ($) = fin de línea
grep 'cat$' /usr/share/dict/words
# Solo palabras que TERMINAN en 'cat'
```

Usar regex en Python con el módulo re

En Python usamos el módulo **re** para trabajar con expresiones regulares:

Ejemplo: Búsqueda básica con re.search()

```
import re

# Buscamos el patrón 'aza' dentro de la palabra 'plaza'
resultado = re.search(r'aza', 'plaza')
# La 'r' delante de la cadena indica raw string (cadena cruda)
# Siempre usa raw strings con regex para evitar confusiones

print(resultado)
# <re.Match object; span=(2, 5), match='aza'>
# span=(2,5) significa que el match está en las posiciones 2 a 5

# Si no hay match, devuelve None:
resultado2 = re.search(r'xyz', 'plaza')
print(resultado2)      # None
```

Caracteres especiales más importantes

Símbolo	Significado	Ejemplo
.	Cualquier carácter (comodín)	p.ng → coincide con 'peng', 'ping', 'pong'...
^	Inicio de línea	^Hola → solo líneas que EMPIEZAN por 'Hola'
\$	Fin de línea	adios\$ → solo líneas que TERMINAN en 'adios'
*	El carácter anterior, 0 o más veces	ab* → 'a', 'ab', 'abb', 'abbb'...
+	El carácter anterior, 1 o más veces (al menos una)	ab+ → 'ab', 'abb', 'abbb'... pero NO 'a'
?	El carácter anterior, 0 o 1 vez (opcional)	colou?r → 'color' o 'colour'
\d	Un dígito (0-9)	\d+ → uno o más números
\w	Letra, número o guion bajo	\w+ → una palabra
\s	Espacio en blanco (espacio, tabulador, salto de línea)	\s+ → uno o más espacios
[abc]	Cualquiera de estos caracteres	[aeiou] → cualquier vocal
[a-z]	Rango de caracteres	[a-zA-Z] → cualquier letra
[^abc]	Cualquier carácter excepto estos	[^0-9] → cualquier cosa que no sea un número
(abc)	Grupo de captura	Captura la parte entre paréntesis
	O (alternativa)	gato perro → 'gato' O 'perro'
\.	Punto literal (escapado)	www\.google\.com → el punto real, no el comodín

Ejemplo real del profesor: extraer IDs de un log

El profesor muestra por qué las regex son mejores que buscar por posición:

Ejemplo: Problema: extraer el ID del proceso de una línea de log

```
# Esta es una línea típica de log:
# 'Jan 31 01:00:10 server kernel[12345]: Error en proceso'

# FORMA MALA (frágil): buscar por posición
```

```
# El ID está en la posición 39... pero ¿y si cambia el formato?
linea = 'Jan 31 01:00:10 server kernel[12345]: Error'
indice = linea.index('(') # Posición del corchete
pid = linea[indice+1:indice+6] # Tomar 5 caracteres
# Problema: ¿y si el ID tiene 6 dígitos? ¿Y si hay otro [ antes?

# FORMA BUENA (robusta): con regex
import re
patron = r'\[(\d+)\]' # Busca números entre corchetes
resultado = re.search(patron, linea)
if resultado:
    pid = resultado.group(1) # El grupo 1 = lo que está entre ()
    print(f'PID: {pid}') # PID: 12345
# Esta regex funciona sin importar cuántos dígitos tenga el ID
```

Grupos de captura: extraer partes específicas

Los paréntesis en regex crean **grupos de captura**. Permiten extraer partes específicas del texto que coincide:

Ejemplo: Ejemplo: reordenar apellido, nombre

```
import re

# Tenemos nombres en formato 'Apellido, Nombre'
# Queremos cambiarlos a 'Nombre Apellido'

def reordenar_nombre(nombre_completo):
    # (\w+) captura el apellido, (\w+) captura el nombre
    resultado = re.search(r'(\w+), (\w+)', nombre_completo)
    if resultado is None:
        return nombre_completo # Si no coincide, lo devuelve tal cual
    # resultado.group(0) = toda la cadena que coincidió
    # resultado.group(1) = primer grupo (apellido)
    # resultado.group(2) = segundo grupo (nombre)
    return f'{resultado.group(2)} {resultado.group(1)}'

print(reordenar_nombre('García, María')) # María García
print(reordenar_nombre('Rodríguez, Juan')) # Juan Rodríguez
```

Funciones del módulo re

Función	¿Qué hace?	Cuándo usarla
<code>re.search()</code>	Busca el patrón en cualquier parte del texto. Devuelve el primer match o None.	Cuando quieres saber SI existe el patrón en el texto
<code>re.findall()</code>	Devuelve una lista con TODOS los matches.	Cuando quieres encontrar todas las ocurrencias
<code>re.split()</code>	Divide el texto usando el patrón como separador.	Cuando quieres separar texto por un patrón flexible
<code>re.sub()</code>	Sustituye las coincidencias por otro texto.	Cuando quieres hacer un buscar-y-reemplazar avanzado

 Para practicar tus regex, el profesor recomienda la web regex101.com. Puedes pegar tu patrón y tu texto, y te muestra exactamente qué coincide y por qué. ¡Muy útil!

Vídeo 5 — Gestión de datos y procesos

Pedir datos al usuario con input()

La función `input()` pausa el programa y espera a que el usuario escriba algo:

Ejemplo: Ejemplo del profesor: conversor de tiempo

```
#!/usr/bin/env python3

def a_segundos(horas, minutos, segundos):
    return horas * 3600 + minutos * 60 + segundos

continuar = True
while continuar:
    horas = int(input('Horas: '))    # input() devuelve texto
    minutos = int(input('Minutos: ')) # int() lo convierte a número
    segundos = int(input('Segundos: '))
    total = a_segundos(horas, minutos, segundos)
    print(f'Total: {total} segundos')
    respuesta = input('¿Otra conversión? (s/n): ')
    continuar = (respuesta == 's')
```

 **input() SIEMPRE devuelve texto (string), incluso si el usuario escribe un número. Por eso el profesor convierte el resultado con `int()` o `float()` cuando necesita operar con números.**

Flujos de entrada/salida: stdin, stdout, stderr

El profesor explica cómo funciona la comunicación entre un programa y el sistema. Existen tres 'canales' de comunicación llamados **I/O streams**:

Canal	Nombre técnico	¿Qué es?	En Python
Entrada	stdin	Los datos que entran al programa (normalmente el teclado)	función <code>input()</code>
Salida normal	stdout	Los resultados que muestra el programa (normalmente la pantalla)	función <code>print()</code>
Salida de errores	stderr	Los mensajes de error (también la pantalla, pero es un canal separado)	errores del intérprete

Estos canales se pueden **redirigir** desde la terminal. Esto es muy útil en automatización:

Ejemplo: Redirigir la salida de un programa

```
# Guardar la salida en un archivo (en vez de mostrarla en pantalla):
python3 mi_script.py > resultado.txt

# Añadir la salida al final de un archivo existente:
python3 mi_script.py >> resultado.txt

# Guardar SOLO los errores en un archivo:
python3 mi_script.py 2> errores.txt

# Usar el contenido de un archivo como entrada del script:
python3 mi_script.py < datos_entrada.txt
```

Variables de entorno

Las variables de entorno son variables que el sistema operativo mantiene y que todos los programas pueden leer. Son como la 'configuración global' del sistema. Puedes verlas desde la terminal con el comando **env**.

La más importante es **PATH**: indica en qué carpetas busca el sistema cuando escribes el nombre de un programa. Cuando escribes python3, el sistema busca en todas las carpetas del PATH hasta encontrarlo.

Ejemplo: Leer variables de entorno desde Python

```
import os

# os.environ es como un diccionario con todas las variables del sistema
# .get() devuelve el valor, o '' si no existe (evita errores)
home = os.environ.get('HOME', '') # Tu carpeta personal
shell = os.environ.get('SHELL', '') # El shell que usas (bash, zsh...)
fruta = os.environ.get('FRUTA', '') # Variable personalizada

print(f'HOME: {home}') # /home/usuario
print(f'SHELL: {shell}') # /bin/bash
print(f'FRUTA: {fruta}') # '' (no está definida todavía)

# Puedes definir tu propia variable de entorno desde la terminal:
# export FRUTA=manzana
# Y luego al ejecutar el script, 'manzana' aparecerá en FRUTA
```

Argumentos de línea de comandos

Puedes pasar información a tu script al ejecutarlo, igual que haces con cualquier comando (por ejemplo, **ls -l carpeta** pasa "-l" y "carpeta" como argumentos). En Python se accede con **sys.argv**:

Ejemplo: Acceder a argumentos del script

```
import sys

# sys.argv es una lista
# sys.argv[0] = nombre del script
# sys.argv[1] = primer argumento
# sys.argv[2] = segundo argumento, etc.

print(sys.argv) # ['mi_script.py', 'argumento1', 'argumento2']

# Ejemplo: script que crea un archivo
nombre_archivo = sys.argv[1] # El usuario pasa el nombre
# Así lo ejecutarías: python3 crear_archivo.py notas.txt
```

Ejecutar comandos del sistema con subprocess

A veces en un script Python necesitas ejecutar un comando del sistema (como si lo escribieras en la terminal). Para eso existe el módulo **subprocess**:

Ejemplo: Ejecutar comandos con subprocess.run()

```
import subprocess

# Ejecutar el comando 'date' (muestra la fecha)
resultado = subprocess.run(['date'])
print(resultado.returncode) # 0 = éxito, otro número = error

# Capturar la SALIDA del comando (en lugar de que se muestre en pantalla)
resultado = subprocess.run(
    ['host', 'www.google.com'],
    capture_output=True,
```

```
        text=True                                # Para obtener texto, no bytes
    )
    print(resultado.stdout)                       # La salida normal del comando
    print(resultado.stderr)                       # Los errores del comando

# Ejemplo con error (archivo que no existe):
resultado = subprocess.run(['ls', 'archivo_inexistente.txt'],
                           capture_output=True, text=True)
print(resultado.returncode)                      # 2 (error)
print(resultado.stderr)                         # 'ls: cannot access ...'
```

Ejemplo real: analizar logs para encontrar problemas

El profesor muestra un ejemplo completo y muy práctico: un servidor se está comportando de forma extraña y sospechan que un cron job (tarea programada) está causando el problema. Quieren saber quién ha estado lanzando cron jobs:

Ejemplo: Script completo: analizar log del servidor

```
#!/usr/bin/env python3
import re
import sys

# Abrir el archivo de log
with open(sys.argv[1]) as f:
    usuarios = {}    # Diccionario para contar ocurrencias

    for linea in f:
        # Solo nos interesan las líneas con 'CRON'
        if 'CRON' not in linea:
            continue    # Saltar esta línea e ir a la siguiente

        # Extraer el nombre del usuario con regex
        # El usuario está al final: ...CMD (nombre_usuario)
        resultado = re.search(r'CMD \((\w+)\)$', linea)
        if resultado is None:
            continue

        usuario = resultado.group(1)
        # Contar cuántas veces aparece cada usuario
        # .get(usuario, 0) devuelve 0 si el usuario no está todavía
        usuarios[usuario] = usuarios.get(usuario, 0) + 1

print(usuarios)    # {'admin': 5, 'root': 3, 'juan': 12}
# ¡Ahora sabemos que 'juan' ha lanzado 12 cron jobs!
```

Vídeo 6 — Testing en Python

¿Por qué necesitamos tests?

El profesor lo explica con una analogía: cuando sacan un coche nuevo de la fábrica, se comprueba que al pisar el acelerador el coche va, y al pisar el freno se para. Con el software es igual: necesitas comprobar que cuando ejecutas el programa, hace lo que se supone que debe hacer.

Hay dos formas de testear:

- **Testing manual:** ejecutas el script a mano con diferentes valores y miras si el resultado es el esperado. Funciona para cosas sencillas, pero es muy difícil de mantener cuando el código crece.
- **Testing automático:** escribes otro código que comprueba automáticamente que tu código funciona bien. El ordenador repite los mismos tests siempre, sin cansarse ni olvidarse de ninguno.

Unit tests: el tipo más común

Los **unit tests** (tests unitarios) comprueban que una función individual funciona correctamente. La idea es simple: dado un input conocido, ¿el output es el esperado?

El profesor usa de ejemplo una función que reordena nombres de 'Apellido, Nombre' a 'Nombre Apellido':

Ejemplo: Crear un test unitario con unittest

```
# archivo: rearrange_test.py
import unittest
from rearrange import rearrange_name    # Importamos la función a testear

class TestRearrange(unittest.TestCase):
    # Cualquier método que empiece por 'test_' se ejecuta como test

    def test_basico(self):
        # Comprobamos que la función hace lo que esperamos
        resultado = rearrange_name('García, María')
        self.assertEqual(resultado, 'María García')    # ¿Son iguales?

    def test_cadena_vacia(self):
        # ¿Qué pasa si le pasamos una cadena vacía?
        resultado = rearrange_name('')
        self.assertEqual(resultado, '')    # Esperamos cadena vacía

    def test_nombre_compuesto(self):
        # ¿Qué pasa con nombres con iniciales o puntos?
        resultado = rearrange_name('Smith, John F.')
        self.assertEqual(resultado, 'John F. Smith')

unittest.main()
```

Para ejecutarlo:

```
python3 rearrange_test.py
..
-----
Ran 2 tests in 0.001s

OK
```

 El profesor explica qué significa la salida: cada punto (.) es un test que ha pasado. Si hay una F, es un test que ha fallado. Al final te dice cuántos tests se ejecutaron y si todo está bien.

Edge cases: los casos extremos que siempre sorprenden

Los **edge cases** son entradas inesperadas o extremas que tu código normalmente no verá, pero que pueden causar errores. El profesor descubre un bug al añadir el test de cadena vacía:

Ejemplo: Descubrir y arreglar un bug con tests


```
# El test falla con este error:
# TypeError: 'NoneType' object is not subscriptable

# ¿Qué está pasando?
# Cuando pasamos '', la regex no encuentra nada y devuelve None.
# El código intenta usar None como si fuera un resultado de búsqueda.

# CÓDIGO CON BUG:
def rearrange_name(nombre):
    resultado = re.search(r'(\w+), (\w+)', nombre)
    return f'{resultado.group(2)} {resultado.group(1)}' # Falla si None

# CÓDIGO ARREGLADO:
def rearrange_name(nombre):
    resultado = re.search(r'(\w+), (\w+)', nombre)
    if resultado is None:
        return nombre # Devolver el original si no hay coincidencia
    return f'{resultado.group(2)} {resultado.group(1)}'
```

💡 *El profesor dice: 'Cuando añades un test y falla, no es mala noticia: ¡acabas de encontrar un bug en tu código! El test hace exactamente lo que debe hacer.' Una vez arreglado el bug, tienes la garantía de que ese mismo error no volverá a aparecer.*

Tipos de tests

Tipo	¿Qué comprueba?	Cuándo usarlo
Unit test	Una función o método aislado.	Siempre. Es el más básico.
Integration test	Que distintas partes del sistema funcionan juntas correctamente.	Cuando tu código interactúa con bases de datos, APIs o servicios externos.
Regression test	Que un bug que ya se arregló no vuelve a aparecer.	Después de arreglar un bug: escribe el test que reproduce el bug, luego arréglalo.
Smoke test	Que el programa básicamente arranca y no explota.	Antes de hacer tests más complejos. ¿Funciona lo más básico?
Load test	Que el sistema aguanta bajo mucha carga.	Antes de lanzar una web o servicio: ¿aguanta 1000 usuarios simultáneos?

Test-Driven Development (TDD)

TDD es una técnica avanzada: escribes los tests **ANTES** de escribir el código. Parece contradictorio, ¿verdad? Pero tiene ventajas:

- Te obliga a pensar bien qué debe hacer tu función antes de escribirla.
- Te hace pensar en qué puede salir mal desde el principio.
- Tienes tests desde el primer momento.

El ciclo de TDD es:

1. Escribes el test (que fallará porque el código no existe todavía).
2. Ejecutas el test y confirmas que falla.
3. Escribes el código mínimo para que el test pase.
4. Ejecutas el test y confirmas que pasa.
5. Repites para la siguiente funcionalidad.

Manejo de errores con try/except

Cuando una operación puede fallar por razones fuera de tu control (el archivo no existe, no tienes permisos, el servidor no responde...), en lugar de dejar que el programa se rompa, puedes capturar el error con **try/except**:

Ejemplo: Capturar errores graciosamente

```
def contar_caracteres(nombre_archivo):
    try:
        with open(nombre_archivo) as f:
            contenido = f.read()
            # Contar cada carácter
            conteo = {}
            for char in contenido:
                conteo[char] = conteo.get(char, 0) + 1
            return conteo
    except OSError as e:
        # OSError cubre: archivo no encontrado, sin permisos, bloqueado...
        print(f'Error al abrir el archivo: {e}')
        return None # Indicamos que la función no pudo completarse

resultado = contar_caracteres('notas.txt')
if resultado is None:
    print('No se pudo leer el archivo.')
else:
    print(resultado)
```

Lanzar tus propios errores con raise

A veces quieres que tu función avise si recibe datos incorrectos. Para eso usas **raise**:

Ejemplo: Validar parámetros y lanzar errores

```
def validar_usuario(nombre, longitud_minima):
    # Si longitud_minima es 0 o negativo, no tiene sentido
    if longitud_minima < 1:
        raise ValueError('longitud_minima debe ser al menos 1')

    if len(nombre) < longitud_minima:
        return False # Nombre demasiado corto

    if not nombre.isalnum():
        return False # Solo se permiten letras y números

    return True

# Esto funciona bien:
print(validar_usuario('maria123', 3)) # True

# Esto lanza un error:
validar_usuario('maria', -1) # ValueError: longitud_minima debe ser...
```



Vídeo 7 — Linux y comandos Bash

Comandos Linux esenciales

El profesor hace un repaso rápido de los comandos más usados en Linux. La filosofía Unix (en la que se basa Linux) es que cada comando debe hacer **una sola cosa y hacerla muy bien**. Por eso hay tantos comandos: cada uno para algo específico.

Comando	¿Qué hace?	Ejemplo
pwd	Muestra en qué carpeta estás ahora mismo	pwd → /home/usuario/proyectos
ls	Lista los archivos de una carpeta	ls -l → lista con detalles (permisos, tamaño, fecha)
cd carpeta	Entra en una carpeta	cd documentos
cd ..	Vuelve a la carpeta anterior	cd .. → sube un nivel
mkdir nombre	Crea una carpeta nueva	mkdir mi_proyecto
rmdir nombre	Borra una carpeta (solo si está vacía)	rmdir carpeta_vacia
cp origen dest	Copia un archivo	cp notas.txt copia_notas.txt
mv origen dest	Mueve o renombra un archivo	mv viejo.txt nuevo.txt
rm archivo	Borra un archivo	rm basura.txt
cat archivo	Muestra el contenido de un archivo	cat notas.txt
touch archivo	Crea un archivo vacío	touch nuevo.txt
echo 'texto'	Imprime texto en la pantalla	echo 'Hola mundo'
chmod +x arch	Da permiso de ejecución al archivo	chmod +x script.py
grep patrón arch	Busca líneas que coincidan con el patrón	grep 'ERROR' log.txt
man comando	Muestra el manual de un comando	man ls

💡 **Los atajos especiales:** el punto (.) representa la carpeta actual, y el doble punto (..) representa la carpeta anterior. Por ejemplo: `cp ../archivo.txt .` → copia el archivo de la carpeta anterior a la carpeta actual.

Detalles del comando ls -l

El comando **ls -l** muestra mucha información. El profesor explica cada columna:

```
-rw-r--r-- 1 usuario grupo 192 Jan 31 10:30 spider.txt
```

Columna	Qué significa
-rw-r--r--	Permisos del archivo (lectura, escritura, ejecución para dueño, grupo y otros)
1	Número de enlaces (no te preocupes por esto ahora)
usuario	Quién es el dueño del archivo
grupo	A qué grupo pertenece
192	Tamaño en bytes

Columna	Qué significa
Jan 31 10:30	Fecha y hora de la última modificación
spider.txt	Nombre del archivo

Redirigir la salida e entrada de comandos

Esta es una de las características más poderosas de la terminal. Puedes hacer que la salida de un comando se guarde en un archivo, o que la entrada de un comando venga de un archivo:

Ejemplo: Redirección de entrada y salida

```
# > redirige stdout a un archivo (sobrescribe si existe)
ls -l > lista_archivos.txt

# >> añade stdout al final de un archivo
echo 'Nueva línea' >> mi_diario.txt

# < usa el contenido de un archivo como entrada
python3 mi_script.py < datos.txt

# 2> redirige solo los errores a un archivo
python3 script_con_errores.py 2> errores.txt

# Los números: 0=stdin, 1=stdout, 2=stderr
```

Pipes: conectar comandos en cadena

El operador | (pipe) conecta la salida de un comando con la entrada del siguiente. Es como una cadena de montaje: cada comando procesa los datos y los pasa al siguiente.

El profesor muestra un ejemplo muy visual para contar las palabras más repetidas en un archivo de texto:

Ejemplo: Pipeline completa: palabras más usadas

```
# Este pipeline encadena 6 comandos:
cat spider.txt | tr ' ' '\n' | sort | uniq -c | sort -nr | head

# Paso a paso:
# cat spider.txt           → muestra el contenido del archivo
# tr ' ' '\n'             → reemplaza espacios por saltos de línea
#                          (una palabra por línea)
# sort                    → ordena alfabéticamente
# uniq -c                 → cuenta las repeticiones y las agrupa
# sort -nr                → ordena numéricamente de mayor a menor
# head                    → muestra solo las 10 primeras líneas

# Resultado: las 10 palabras más usadas con su número de apariciones
```

 *El profesor dice: 'No hace falta memorizar todos estos comandos. Lo importante es saber que podemos encadenarlos. Cuando necesites hacer algo, buscas qué comandos existen para esa tarea y los combinas.'*

Señales: comunicarse con procesos en ejecución

Cuando un programa está ejecutándose, puedes enviarle señales para controlarlo:

Tecla / Comando	Señal que envía	Efecto
Ctrl + C	SIGINT	Interrumpe el programa. El programa termina limpiamente (puede mostrar un resumen antes de salir).
Ctrl + Z	SIGSTOP	Pausa el programa (no termina, solo se pausa). Usa 'fg' para reanudar.
kill PID	SIGTERM	Termina el proceso con ese ID. Más brusco que Ctrl+C.

Para encontrar el PID (identificador) de un proceso y enviarle una señal:

```
ps ax | grep ping      # Buscar el PID del proceso ping
# Resultado: 4619 pts/0 S    0:00 ping www.google.com
kill 4619              # Terminar ese proceso
```

Vídeo 8 — Scripts Bash y cuándo usar Bash vs Python

¿Qué es Bash y por qué aprenderlo si ya sé Python?

El profesor explica que Bash no es solo la terminal: es también un lenguaje de programación. Y aunque ya sabemos Python, Bash es útil para tareas sencillas con comandos del sistema.

 *Ejemplo del profesor: si quieres convertir 500 imágenes PNG a JPEG, en Python necesitarías importar os, iterar los archivos, llamar subprocess... En Bash son 3 líneas. Para tareas simples con archivos y comandos del sistema, Bash es más rápido y legible.*

Tu primer script Bash

El profesor muestra un script que recoge información del sistema (útil para depurar un ordenador que se comporta mal):

Ejemplo: Script de diagnóstico del sistema

```
#!/bin/bash
# La primera línea es el shebang para Bash (igual que en Python)

# Definimos una variable con una línea de guiones
LINEA='-----'

echo '=== DIAGNÓSTICO DEL SISTEMA ==='
echo "Fecha: $(date)"      # $(comando) ejecuta el comando y usa su salida
echo $LINEA

echo '--- Tiempo encendido ---'
uptime
echo $LINEA

echo '--- Memoria libre ---'
free
echo $LINEA

echo '--- Usuarios conectados ---'
who
```

Para ejecutarlo:

```
chmod +x diagnostico.sh
./diagnostico.sh
```

Variables en Bash

Similar a Python pero con una diferencia crítica: **no puede haber espacios alrededor del =**:

Ejemplo: Variables en Bash: reglas importantes

```
# CORRECTO:
nombre='Ana'
edad=25

# INCORRECTO (dará error):
nombre = 'Ana'      # Error: intenta ejecutar el comando 'nombre'

# Para usar el valor de una variable, pon $ delante:
echo "Hola, $nombre"      # Hola, Ana
echo "Tienes $edad años"

# Si quieres que otros comandos vean la variable, usa export:
export MI_VARIABLE='valor'
```

Globs: crear listas de archivos

Los globs son caracteres especiales para seleccionar grupos de archivos:

Ejemplo: Globs para seleccionar archivos

```
# * (estrella) = cualquier cantidad de caracteres
ls *.py          # Todos los archivos que terminan en .py
ls datos_*       # Todos los que empiezan por 'datos_'
ls *             # Todos los archivos del directorio

# ? (interrogación) = exactamente UN carácter
ls ?????.py      # Scripts Python con 5 caracteres en el nombre
# Ej: areas.py, hello.py, clean.py
```

Condicionales en Bash: if/then/else/fi

Los if en Bash funcionan diferente a Python: en vez de comprobar True/False, comprueban el **código de salida** de un comando (0 = éxito = condición verdadera):

Ejemplo: Condicional con grep

```
#!/bin/bash
# Comprobamos si 127.0.0.1 está en el archivo /etc/hosts

if grep '127.0.0.1' /etc/hosts; then
    echo 'OK: La entrada 127.0.0.1 está en /etc/hosts'
else
    echo 'PROBLEMA: Falta la entrada 127.0.0.1'
fi      # Nota: if al revés. Así termina el bloque if en Bash.

# También puedes usar el comando 'test' con corchetes:
if [ -n "$ruta" ]; then      # -n comprueba si la variable NO está vacía
    echo "La ruta es: $ruta"
fi
```

Bucles en Bash

Bucle while

Ejemplo: Bucle while: contar del 1 al 5

```
#!/bin/bash
n=1
while [ $n -le 5 ]; do    # -le significa 'menor o igual que'
    echo "Intento número $n"
    n=$((n + 1))         # Así hacemos aritmética en Bash
done
```

El profesor muestra un ejemplo muy práctico: un script que reintenta un comando si falla (útil cuando el fallo puede ser temporal, por ejemplo por problemas de red):

Ejemplo: Script de reintento automático

```
#!/bin/bash
# Reintenta un comando hasta que funcione o falle 5 veces

comando=$1    # $1 = primer argumento que pasamos al script
intentos=0

while ! $comando; do    # ! invierte el resultado: entra si el comando FALLA
    sleep $intentos     # Espera más tiempo en cada reintento
    intentos=$((intentos + 1))
    if [ $intentos -ge 5 ]; then    # -ge = mayor o igual que
        echo 'El comando falló 5 veces. Abandono.'
        break
    fi
    echo "Reintento número $intentos..."
done

# Uso: ./reintento.sh mi_comando
```

Bucle for con globs: renombrar archivos en masa

El profesor muestra un ejemplo muy real: necesitas renombrar 500 archivos .HTM a .html porque el nuevo servidor web los requiere en minúsculas:

Ejemplo: Renombrar archivos en masa

```
#!/bin/bash
# Renombrar todos los .HTM a .html

for archivo in *.HTM; do    # Itera sobre todos los archivos .HTM
    # basename extrae el nombre sin la extensión
    nombre=$(basename "$archivo" .HTM)

    # mv renombra el archivo
    mv "$archivo" "$nombre.html"
done

# TRUCO DEL PROFESOR: antes de ejecutarlo de verdad,
# pon 'echo' delante del mv para ver qué haría SIN hacerlo:
# echo mv "$archivo" "$nombre.html"
# Así comprobamos que funciona correctamente antes de tocar nada.
```

¿Cuándo usar Bash y cuándo usar Python?

Esta es la pregunta clave del módulo. El profesor da una guía clara:

Usa Bash cuando...	Usa Python cuando...
La tarea es sencilla y usa comandos del sistema	La lógica es compleja (muchos condicionales, bucles anidados)
Solo trabajas en Linux/Mac (no necesitas Windows)	Necesitas que funcione en varios sistemas operativos
El script son pocas líneas y se entiende fácilmente	El script es largo y necesita ser legible y mantenible
Principalmente mueves, copias o renombas archivos	Necesitas procesar texto, listas, diccionarios, datos
Combinas varios comandos del sistema existentes	No existe un comando del sistema para lo que necesitas

💡 *El profesor dice: 'Cuando el Bash empieza a ponerse tan complejo que cuesta entenderlo, es la señal de que debes pasarte a Python. Un script de Bash con 50 líneas llenas de if y variables raras es mucho más difícil de mantener que el equivalente en Python.'*



Mapa completo del curso

Vídeo	Tema	Conceptos clave
1	Instalar Python	SO, kernel, pip, PyPI, Python 3, instalar módulos
2	Ejecutar scripts	Interpretado vs compilado, .py, shebang, chmod, módulos propios, cuándo automatizar
3	Archivos	open(), with, readline, write, modos r/w/a, os module, CSV
4	Regex	Patrones, re.search, caracteres especiales, grupos de captura, re.sub
5	Datos y procesos	input(), stdin/stdout/stderr, variables de entorno, subprocess, logs
6	Testing	unittest, assertEquals, edge cases, try/except, raise, TDD
7	Linux y Bash	Comandos ls/cp/mv/rm, redirección, pipes, señales
8	Scripts Bash	Variables, globs, if/fi, while/do/done, for, Bash vs Python



Glosario de términos

Término	Explicación sencilla
Script	Un archivo .py con instrucciones Python que se ejecuta desde la terminal.
Módulo	Un archivo .py que puedes importar en otro script para reutilizar su código.
pip	La herramienta para instalar módulos externos de Python desde PyPI.
PyPI	El 'repositorio' oficial de módulos Python. Como una tienda de librerías gratuita.
Shebang	La línea #!/usr/bin/env python3 al inicio de un script. Le dice al SO cómo ejecutarlo.

Término	Explicación sencilla
chmod +x	Comando Linux para dar permisos de ejecución a un archivo.
Intérprete	El programa que lee y ejecuta el código Python línea a línea.
with open()	Forma segura de abrir archivos: los cierra automáticamente al terminar el bloque.
Modo 'w'	Modo escritura de archivos. Cuidado: BORRA el contenido existente al abrir.
Modo 'a'	Modo append (añadir). Conserva el contenido y añade al final.
os módulo	Módulo de Python para gestionar archivos y carpetas desde el código.
CSV	Formato de archivo: Comma Separated Values. Como una hoja de cálculo en texto plano.
Regex	Expresión regular: un patrón para buscar texto de forma flexible y potente.
stdin/stdout/stderr	Los tres canales de comunicación de un programa con el sistema.
subprocess	Módulo Python para ejecutar comandos del sistema desde un script.
Unit test	Test que comprueba que una función individual funciona correctamente.
Edge case	Caso extremo o inesperado que puede hacer que tu código falle.
try/except	Bloque para capturar errores y gestionarlos sin que el programa se rompa.
raise	Palabra clave para lanzar un error desde tu propio código.
Pipe ()	Conecta la salida de un comando con la entrada del siguiente en la terminal.
Redirección (>)	Envía la salida de un comando a un archivo en lugar de a la pantalla.
Bash	El lenguaje de programación de la terminal Linux. Ideal para scripts cortos con comandos.
Glob	Caracteres especiales (* y ?) para seleccionar grupos de archivos por nombre.
Exit status	Código que devuelve un programa al terminar. 0 = éxito. Otro = error.